

Assignment-8

2303A51635

S. Keerthi

Batch-23

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.

- Requirements:

- o Username length must be between 5 and 15 characters.
- o Must contain only alphabets and digits.
- o Must not start with a digit.
- o No spaces allowed.

Example Assert Test Cases:

```
assert is_valid_username("User123") == True
```

```
assert is_valid_username("12User") == False
```

```
assert is_valid_username("Us er") == False
```

Expected Output #1:

- Username validation logic successfully passing all AI-generated test cases.

```
2303A51666_LabAssignment-8.py > ...
1  def is_valid_username(username):
2      if len(username) < 5 or len(username) > 15:
3          return False
4      if not username[0].isalpha():
5          return False
6      for char in username:
7          if not (char.isalnum() or char == '_'):
8              return False
9      return True
10 #test cases for the is_valid_username function
11 assert is_valid_username("User123") == True
12 assert is_valid_username("12User") == False
13 assert is_valid_username("Us er") == False
14 print("All test cases for is_valid_username passed!")
```

```
2303A51666_LabAssignment-8.py
All test cases for is valid username passed!
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.

- Requirements:

- o If input is an integer, classify as "Even" or "Odd".

- o If input is 0, return "Zero".

- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

```
assert classify_value(8) == "Even"
```

```
assert classify_value(7) == "Odd"
```

```
assert classify_value("abc") == "Invalid Input"
```

Expected Output #2:

- Function correctly classifying values and passing all test cases.

```
17 def classify_value(x):
18     if x < 0:
19         return "Negative"
20     elif x == 0:
21         return "Zero"
22     elif x%2 == 0:
23         return "Even"
24     else:
25         return "Odd"
26 # Test cases for the classify_value function
27 assert classify_value(8) == "Even"
28 assert classify_value(7) == "Odd"
29 assert classify_value("abc") == "Invalid Input"
```

```
if x < 0:
    ^^^^^
TypeError: '<' not supported between instances of 'str' and 'int'
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.

- Requirements:

- o Ignore case, spaces, and punctuation.

- o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

`assert is_palindrome("Madam") == True`

`assert is_palindrome("A man a plan a canal Panama") == True`

`assert is_palindrome("Python") == False`

```
31 def is_palindrome(text):
32     cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
33     return cleaned_text == cleaned_text[::-1]
34 # Test cases for the is_palindrome function
35 assert is_palindrome("Madam") == True
36 assert is_palindrome("A man a plan a canal Panama") == True
37 assert is_palindrome("Python") == False
38 print("All test cases for is_palindrome passed!")
```

All test cases for is_palindrome passed!

Task Description #4 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.

- Requirements:

- o Must contain @ and .

- o Must not start or end with special characters.

- o Should handle invalid formats gracefully.

Example Assert Test Cases:

`assert validate_email("user@example.com") == True`

`assert validate_email("userexample.com") == False`

`assert validate_email("@gmail.com") == False`

Expected Output #5:

- Email validation function passing all AI-generated test cases and handling edge cases correctly.

```

68 def validate_email(email):
69     if '@' not in email or '.' not in email:
70         return False
71     at_index = email.index('@')
72     dot_index = email.rindex('.')
73     if at_index < 1 or dot_index < at_index + 2 or dot_index >= len(email) - 1:
74         return False
75     return True
76 # Test cases for the validate_email function
77 assert validate_email("user@example.com") == True
78 assert validate_email("userexample.com") == False
79 assert validate_email("@gmail.com") == False
80 print("All test cases for validate_email passed!")

```

All test cases for validate_email passed!

Task 5 (Perfect Number Checker – Test Case Design)

- Function: Check if a number is a perfect number (sum of divisors = number).
- Test Cases to Design:
 - o Normal case: 6 → True, 10 → False.
 - o Edge case: 1.
 - o Negative number case.
 - o Larger case: 28.
- Requirement: Validate correctness with assertions.

```

83 #generate a python code to display whether the given number is perfect number or not
84 def is_perfect_number(num):
85     if num < 1:
86         return False
87     divisors_sum = sum(i for i in range(1, num) if num % i == 0)
88     return divisors_sum == num
89 # Test cases for the is_perfect_number function
90 assert is_perfect_number(6) == True
91 assert is_perfect_number(10) == False
92 assert is_perfect_number(28) == True
93 assert is_perfect_number(1) == False
94 print("All test cases for is_perfect_number passed!")

```

All test cases for is_perfect_number passed!

Task 6 (Abundant Number Checker – Test Case Design)

- Function: Check if a number is abundant (sum of divisors > number).
- Test Cases to Design:
 - o Normal case: 12 → True, 15 → False.
 - o Edge case: 1.
 - o Negative number case.

o Large case: 945.

Requirement: Validate correctness with unittest

```
50 def Abundant_Number_Checker(num):
51     if num < 1:
52         return False
53     divisors_sum = sum(i for i in range(1, num) if num % i == 0)
54     return divisors_sum > num
55 import unittest
56 class TestAbundantNumberChecker(unittest.TestCase):
57     def test_abundant(self):
58         self.assertTrue(Abundant_Number_Checker(12))
59         self.assertTrue(Abundant_Number_Checker(15))
60         self.assertTrue(Abundant_Number_Checker(1))
61         self.assertFalse(Abundant_Number_Checker(-1))
62         self.assertTrue(Abundant_Number_Checker(945))
63 if __name__ == '__main__':
64     unittest.main()

self.assertTrue(Abundant_Number_Checker(15))
~~~~~
AssertionError: False is not true

-----
Ran 1 test in 0.001s

FAILED (failures=1)
```

Task 7 (Deficient Number Checker – Test Case Design)

- Function: Check if a number is deficient (sum of divisors < number).

- Test Cases to Design:

- o Normal case: 8 → True, 12 → False.

- o Edge case: 1.

- o Negative number case.

- o Large case: 546.

Requirement: Validate correctness with pytest

```

77  def deficient_number_checker(num):
78      if num < 1:
79          return False
80      divisors_sum = sum(i for i in range(1, num) if num % i == 0)
81      return divisors_sum < num
82  def test_deficient_number_checker():
83      assert deficient_number_checker(8) == True
84      assert deficient_number_checker(12) == False
85      assert deficient_number_checker(1) == True
86      assert deficient_number_checker(546) == False
87      print("All test cases for deficient_number_checker passed!")

```

plugins: anyio-4.10.0
collected 1 item

lab-081.py .

===== 1 passed in 0.06s =====

Task 8 :

Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

```

89  def LeapYearChecker(year):
90      if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
91          return True
92      return False
93  def test_leap_year_checker():
94      assert LeapYearChecker(2020) == True
95      assert LeapYearChecker(1900) == False
96      assert LeapYearChecker(2000) == True
97      assert LeapYearChecker(2021) == False
98      assert LeapYearChecker(2400) == True
99      assert LeapYearChecker(2100) == False
100     assert LeapYearChecker(1996) == True
101     assert LeapYearChecker(1999) == False
102     assert LeapYearChecker(1600) == True
103     print("All test cases for LeapYearChecker passed!")

```

plugins: anyio-4.10.0
collected 1 item

lab-081.py .

===== 1 passed in 0.09s =====

Task 9 :

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

```

105 def SumOfDigits(num):
106     return sum(int(digit) for digit in str(abs(num)))
107 def test_sum_of_digits():
108     assert SumOfDigits(123) == 6
109     assert SumOfDigits(-456) == 15
110     assert SumOfDigits(0) == 0
111     assert SumOfDigits(9999) == 36
112     assert SumOfDigits(-1001) == 2
113     print("All test cases for SumOfDigits passed!")

```

plugins: anyio-4.10.0
collected 1 item

lab-081.py .

===== 1 passed in 0.05s =====

Task 10 :

Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

```

115 def SortNumbers(numbers):
116     n= len(numbers)
117     for i in range(n):
118         for j in range(0, n-i-1):
119             if numbers[j] > numbers[j+1]:
120                 numbers[j], numbers[j+1] = numbers[j+1], numbers[j]
121     return numbers
122 def test_sort_numbers():
123     assert SortNumbers([5, 2, 9, 1, 5, 6]) == [1, 2, 5, 5, 6, 9]
124     assert SortNumbers([]) == []
125     assert SortNumbers([3]) == [3,5]
126     assert SortNumbers([3, 2]) == [2, 3]
127     assert SortNumbers([1, 2, 3]) == [1, 2, 3]
128     assert SortNumbers([3, 2, 1]) == [1, 2, 3]
129     assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
130     assert SortNumbers([1, 1, 1, 1]) == [1, 1, 1, 1]
131     assert SortNumbers([9, 8, 7, 6, 5]) == [5, 6, 7, 8, 9]
132     assert SortNumbers([10, 9, 8, 7, 6]) == [6, 7, 8, 9, 10]
133     assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
134     assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
135     assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
136     assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
137     assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
138     print("All test cases for SortNumbers passed!")

```



```
===== FAILURES =====
test_sort_numbers

def test_sort_numbers():
    assert SortNumbers([5, 2, 9, 1, 5, 6]) == [1, 2, 5, 5, 6, 9]
    assert SortNumbers([]) == []
>   assert SortNumbers([3]) == [3,5]
E   assert [3] == [3, 5]
E
E       Right contains one more item: 5
E       Use -v to get more diff

lab-081.py:125: AssertionError
===== short test summary info =====
FAILED lab-081.py::test_sort_numbers - assert [3] == [3, 5]
===== 1 failed in 0.22s =====
```

Task 11 :

Write a function ReverseString and validate its implementation using 5 unittest test cases

```
140 def ReverseString(s):
141     return s[::-1]
142 import unittest
143 class TestReverseString(unittest.TestCase):
144     def test_reverse_string(self):
145         self.assertEqual(ReverseString("Hello"), "olleH")
146         self.assertEqual(ReverseString("Python"), "nohtyP")
147         self.assertEqual(ReverseString(""), "")
148         self.assertEqual(ReverseString("A"), "A")
149         self.assertEqual(ReverseString("12345"), "54321")
150 if __name__ == '__main__':
151     unittest.main()
```

```
op/AI ASSISTANT CODING/lab-081.py"
.
-----
Ran 1 test in 0.001s

OK
```

Task 12 :

Write a function AnagramChecker and validate its implementation using 10 unittest test cases.


```

153 v def AnagramChecker(str1,str2):
154     return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
155     import unittest
156 v class TestAnagramChecker(unittest.TestCase):
157 v     def test_anagram_checker(self):
158         self.assertTrue(AnagramChecker("listen", "silent"))
159         self.assertTrue(AnagramChecker("Triangle", "Integral"))
160         self.assertFalse(AnagramChecker("Hello", "World"))
161         self.assertTrue(AnagramChecker("Dormitory", "Dirty Room"))
162         self.assertFalse(AnagramChecker("Python", "Java"))
163         self.assertTrue(AnagramChecker("State", "Taste"))
164         self.assertTrue(AnagramChecker("Conversation", "Voices Rant On"))
165         self.assertFalse(AnagramChecker("Apple", "Appeal"))
166         self.assertTrue(AnagramChecker("Astronomer", "Moon Starer"))
167         self.assertFalse(AnagramChecker("Earth", "Hearts"))
168 v if __name__ == '__main__':
169     unittest.main()

```

```

op/AI ASSISTANT CODING/lab-081.py
op/AI ASSISTANT CODING/lab-081.py
.
-----
Ran 1 test in 0.000s

OK

```

Task 13 :

Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.

```

172 def ArmstrongChecker(num):
173     num_str = str(num)
174     num_digits = len(num_str)
175     armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
176     return armstrong_sum == num
177 import unittest
178 class TestArmstrongChecker(unittest.TestCase):
179     def test_armstrong_checker(self):
180         self.assertTrue(ArmstrongChecker(153))
181         self.assertTrue(ArmstrongChecker(9474))
182         self.assertFalse(ArmstrongChecker(123))
183         self.assertTrue(ArmstrongChecker(0))
184         self.assertTrue(ArmstrongChecker(1))
185         self.assertFalse(ArmstrongChecker(10))
186         self.assertTrue(ArmstrongChecker(375))
187         self.assertTrue(ArmstrongChecker(371))
188 if __name__ == '__main__':
189     unittest.main()

```

```

self.assertTrue(ArmstrongChecker(375))
~~~~~
AssertionError: False is not true

-----
Ran 1 test in 0.003s

FAILED (failures=1)

```